

Employee Management System — Functional Facilities

#	Functional Facility	What it does	Why we need it
1	Department Management (CRUD)	Create, view, update and delete departments	Employee needs to belong to a department, so we establish this first
2	Create Employee (with Department)	Add a new employee and assign a department	Main functionality of the system
3	Get Employee by ID	Get complete details of one employee	Used when we need a specific employee
4	Get All Employees	Get all employees	Allows HR/admin to view the employee list
5	Update Employee	Modify employee information	Employee details can change
6	Delete Employee (Hard Delete)	Permanently remove an employee record	Teaches the basic delete operation first
7	Employee Status + Soft Delete	Manage ACTIVE, INACTIVE, TERMINATED employees	Allows us to deactivate employees without immediately removing their records
8	Search Employees	Search by name, email or phone	Helps find employees quickly
9	Filter Employees	Filter by department, status or gender	Helps narrow down employee records
10	Pagination	Return employees page by page	Prevents loading a large number of employees at once
11	Sorting	Sort employees by name, salary, hire date, etc.	Makes employee data easier to organize
12	Basic Reports	Employee counts/details by department or status	Provides useful management information

How we will build it

Phase 1 — Foundation

1. Department CRUD



2. Employee Create



3. Employee Get By ID



4. Employee Get All



5. Employee Update



6. Employee Hard Delete

First, we establish the basic working system.

Phase 2 — Employee Lifecycle

7. Employee Status



Soft Delete

Now we handle what happens when an employee is no longer active

Phase 3 — Data Retrieval

8. Search



9. Filter



10. Pagination



11. Sorting

These will progressively make:

GET /employees

more powerful.

For example:

GET /employees

then eventually:

GET /employees?name=Ram

GET /employees?department=IT&status=ACTIVE

GET /employees?page=0&size=10

GET /employees?sort=name,asc

We don't need to create a completely different endpoint for every one of these.

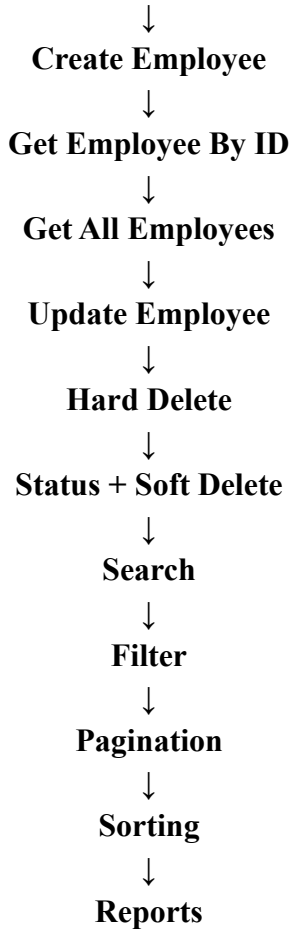
Phase 4 — Reporting

12. Basic Reports

Reports come last because they depend on the employee and department data already being properly established.

Final order

Department CRUD



Employee Management System — Technical Facilities

#	Technical Facility	What it does	Why we need it
1	Maven & Dependencies	Manages project libraries and dependencies such as Spring Boot, JPA and MySQL	Provides the required libraries and build system
2	Layered Architecture	Separates Controller, Service, Repository and Entity responsibilities	Keeps the application clean, maintainable and loosely coupled
3	Spring Boot Configuration + Environment Variables	Configures application settings and keeps sensitive values such as DB credentials outside source code	Properly configures the application and protects sensitive configuration
4	Dependency Injection	Spring provides required objects/Beans to classes	Reduces manual object creation and promotes loose coupling
5	REST API Architecture	Uses controllers, HTTP methods, endpoints and HTTP status codes	Allows Postman/frontend clients to communicate with the backend
6	DTO + Manual Mapper	Converts Request DTO ↔ Entity ↔ Response DTO	Separates API data from database entities and prevents direct entity exposure
7	JPA / Hibernate + Spring Data Repository	Maps Java objects to MySQL tables and performs database operations	Simplifies database access compared with manual JDBC code
8	Entity Relationships	Defines relationships such as Employee ↔ Department	Represents real business relationships in the database
9	Transactions (@Transactional)	Groups database operations into a transaction	Helps maintain database consistency during create, update and delete operations
10	Pagination & Sorting (Pageable, Sort)	Retrieves data page-by-page and in a specified order	Makes employee retrieval efficient and scalable
11	Logging	Records important application activities, warnings and errors	Helps monitor, debug and understand application behavior

#	Technical Facility	What it does	Why we need it
12	Swagger / OpenAPI	Documents and displays REST APIs	Makes APIs easier for developers to understand and test
13	Unit & Integration Testing	Tests individual components and complete application flows	Helps verify correctness and prevent regressions
14	Security	Provides authentication, authorization, roles and password hashing	Protects employee data and controls access
15	Production Configuration	Uses profiles, Actuator and production-oriented configuration	Makes the application easier to monitor and operate in a real environment

How We Will Implement Them

Phase 0 — Project Setup

Maven & Dependencies



Layered Architecture



Spring Boot Configuration



Environment Variables



Dependency Injection

This prepares the project before implementing business functionality.

Phase 1 — Department CRUD

REST API



DTO + Manual Mapper



JPA / Hibernate



Repository



Logging

Functional facility: Department Management (CRUD)

At this stage we also introduce:

Validation

Exception Handling

so we learn them early rather than adding them at the end.

Phase 2 — Employee Foundation

Entity Relationships

↓

Employee ↔ Department

Then:

Create Employee

Get Employee

Get All Employees

Update Employee

Hard Delete

The existing REST, DTO, Mapper, JPA, Repository, Validation, Exception Handling and Logging concepts are reused.

Phase 3 — Employee Lifecycle

Transactions

↓

Employee Status + Soft Delete

Example:

ACTIVE

INACTIVE

TERMINATED

Phase 4 — Data Retrieval

Search



Filter



Pagination



Sorting

Using:

Pageable

Sort

Spring Data JPA queries

These progressively enhance:

GET /employees

Phase 5 — Reports

Repository Queries



Service



DTO



REST API

For example:

Employees by Department

Employees by Status

Employee counts

Phase 6 — Documentation & Testing

Swagger / OpenAPI



Unit Testing



Integration Testing

Phase 7 — Security

Authentication



Authorization



Roles



Password Hashing

Security will be added after the core Employee Management functionality.

Phase 8 — Production Configuration

Profiles



Environment Configuration

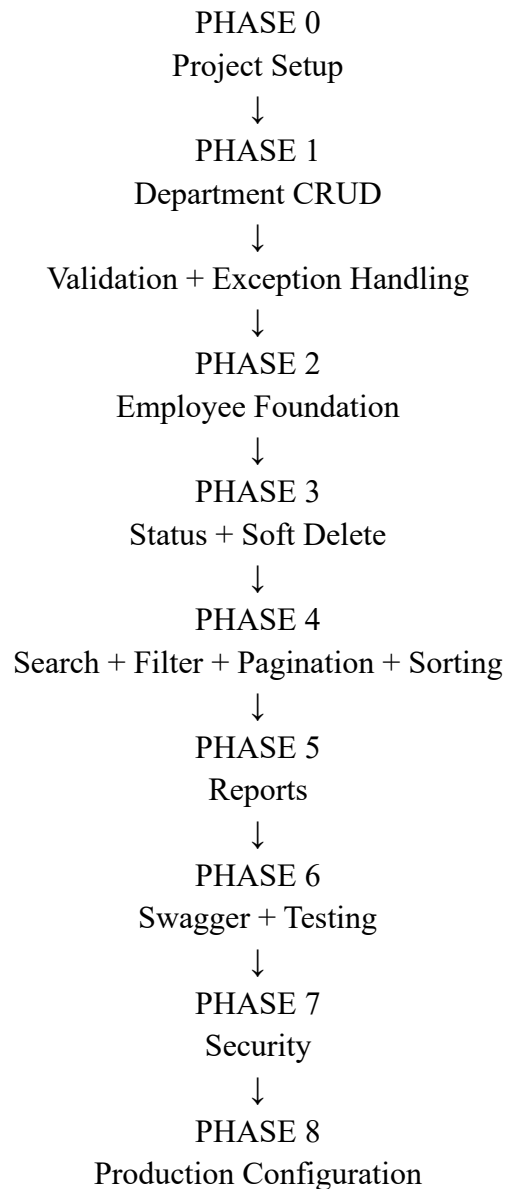


Actuator



Production Settings

Final Development Flow



Important

Validation and **Exception Handling** remain your separate categories, not Technical Facilities. **Git** is also not a numbered facility. We use it continuously:

Code
↓
Test
↓
Commit
↓
Next feature

Employee Management System —Validation Facilities

Validation Categories

#	Validation Category	Where	Tool / Approach	Why we need it
1	Input Validation	Request DTO	Bean Validation annotations	Checks the format, required values and basic limits of incoming data
2	Business Validation	Service Layer	Service logic + custom exceptions	Checks rules that require database data or current business state
3	Database Data Integrity	MySQL	UNIQUE, NOT NULL, FOREIGN KEY, etc.	Final protection against inconsistent or duplicate data
4	Parameter Validation	Controller / Service / Configuration	Limits + allowed-field checks	Protects search, filter, pagination and sorting parameters

1. Input Validation

Where: Request DTO

Tool: Jakarta Bean Validation

These validations check the **shape and basic value** of incoming data.

Department

Field	Validation
name	@NotBlank, @Size
description	@Size(max = ...)

Example:

@NotBlank

@Size(min = 2, max = 50)

private String name;

Employee

Field	Validation
name	@NotBlank, @Size
email	@NotBlank, @Email, @Size
phone	@NotBlank, @Pattern, @Size
address	@NotBlank, @Size
gender	@NotNull
dateOfBirth	@NotNull, @Past
hireDate	@NotNull, @PastOrPresent
employeeType	@NotNull
departmentId	@NotNull, @Positive
baseSalary	@NotNull, @PositiveOrZero, @Digits

Employee status

For **Create Employee**:

status

↓

NOT provided by client

↓

Server sets ACTIVE

So we **do not use @NotNull** for status on the Create DTO.

Update Employee

The **Update DTO** uses the same field-level validation rules as Create.

For example:

name → @NotBlank, @Size

email → @NotBlank, @Email, @Size

phone → @NotBlank, @Pattern, @Size

salary → @NotNull, @PositiveOrZero, @Digits

But uniqueness is handled differently.

For email:

Create:

existsByEmail(email)

Update:

existsByEmailAndIdNot(email, id)

Why?

Because an employee should be allowed to update other information without getting:

409 Email already exists

for their **own existing email**.

2. Business Validation

Where: Service Layer

Tool: Business logic + custom exceptions

These rules cannot be solved by annotations alone because they require database/state information.

#	Business Rule	Result
1	Department name must be unique	409 Conflict
2	Employee email must be unique	409 Conflict
3	departmentId must exist	404 Not Found
4	Department cannot be deleted while employees belong to it	409 Conflict
5	Employee status must follow allowed transitions	409 Conflict
6	Employee must satisfy minimum age requirement	400 Bad Request

Status transitions

Our rule will be:

ACTIVE

↓

INACTIVE

TERMINATED

INACTIVE

↓

ACTIVE

TERMINATED

TERMINATED

↓

Nothing

So:

TERMINATED → ACTIVE

is not allowed.

This is **business validation**, because we need to know the employee's current status.

Minimum age

We use:

@Past

for the basic date rule.

But @Past only means:

DOB must be before today.

It does **not** mean:

Employee must be at least 18 years old.

Therefore:

@Past

+

minimum age business rule

The minimum age rule belongs in **Business Validation**.

3. Database Data Integrity

Where: MySQL

Tool: Database constraints

The database is our **final safety layer**.

Important constraints

Employee.email

↓

UNIQUE

Department.name

↓

UNIQUE

Employee.department_id

↓

FOREIGN KEY

↓

Department.id

Required database fields can also use:

NOT NULL

Why do we need database constraints if Service already checks?

Because of concurrent requests.

For example:

Request A → email doesn't exist

Request B → email doesn't exist

Both can pass the Service check at almost the same time.

The database UNIQUE constraint provides the final protection.

If the database rejects the operation:

DataIntegrityViolationException



Global Exception Handler



409 Conflict

4. Parameter Validation

This applies to:

Search

Filter

Pagination

Sorting

Pagination

Example:

GET /employees?page=0&size=10

Rules:

$\text{page} \geq 0$

$\text{size} > 0$

$\text{size} \leq \text{configured maximum}$

We should also configure a maximum page size.

For example:

size = 10 

size = 50 

size = 100000 

This prevents unnecessarily large database/API responses.

Sorting

Example:

GET /employees?sort=name,asc

We should maintain an **allowed sorting-field list**.

For example:

name

salary

hireDate

An arbitrary field should not be allowed to reach JPA/Hibernate.

Search / Filter

Examples:

GET /employees?name=Ram

GET /employees?department=IT&status=ACTIVE

We validate relevant values where necessary, without adding unnecessary validation to every query parameter.

Validation + Exception Handling

These are separate responsibilities.

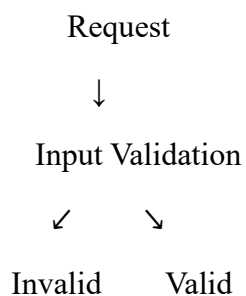
Validation

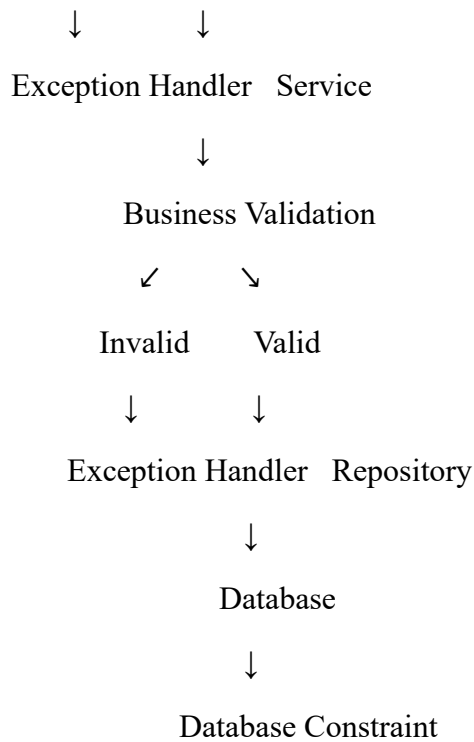
Finds the problem.

Exception Handling

Converts the problem into a proper API response.

The complete flow is:





So input validation errors, business validation errors, and database constraint errors can all eventually be handled by our Global Exception Handler, but they originate at different layers.

Validation Annotations We Need

For this project, our main Bean Validation toolkit is:

@NotNull

@NotBlank

@Size

@Email

@Pattern

@Positive

@PositiveOrZero

@Digits

@Past

@PastOrPresent

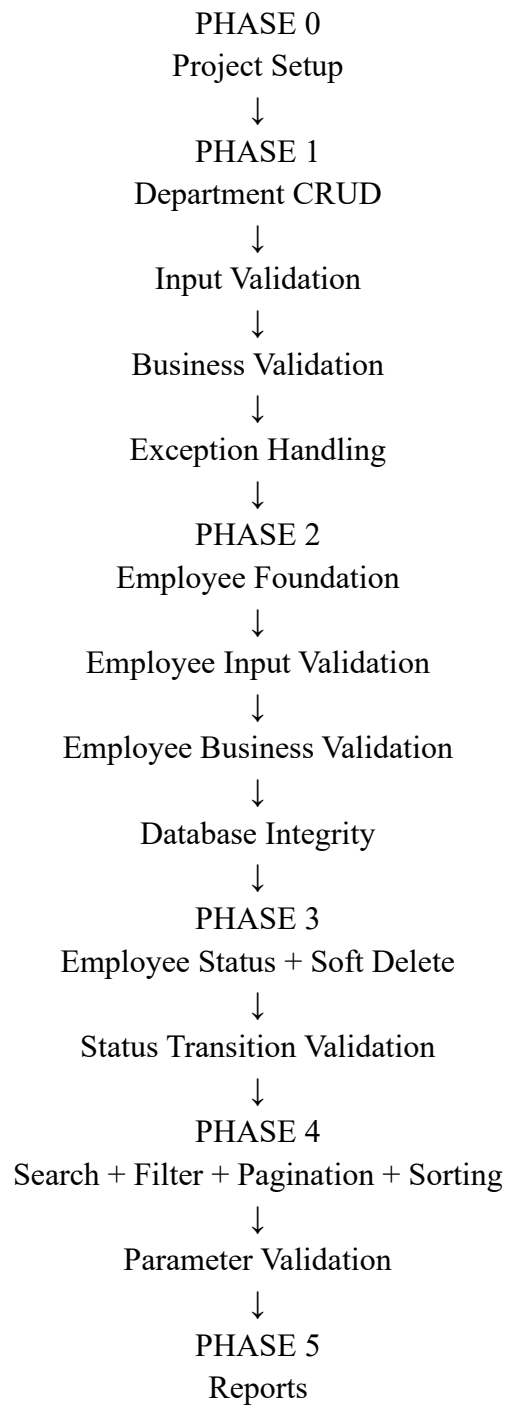
We will **not use an annotation just because it exists.**

For every field, we ask:

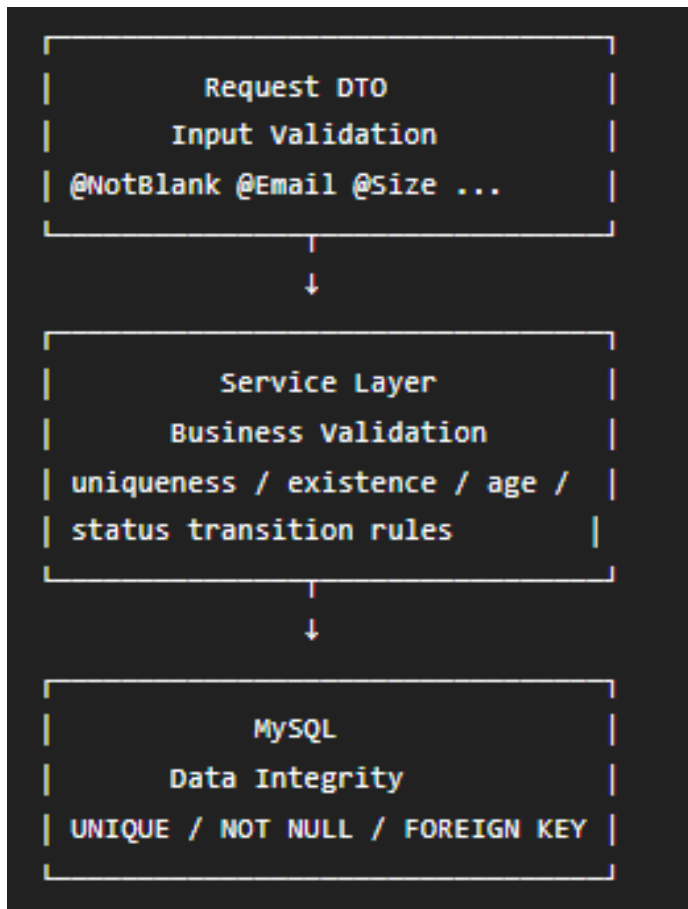
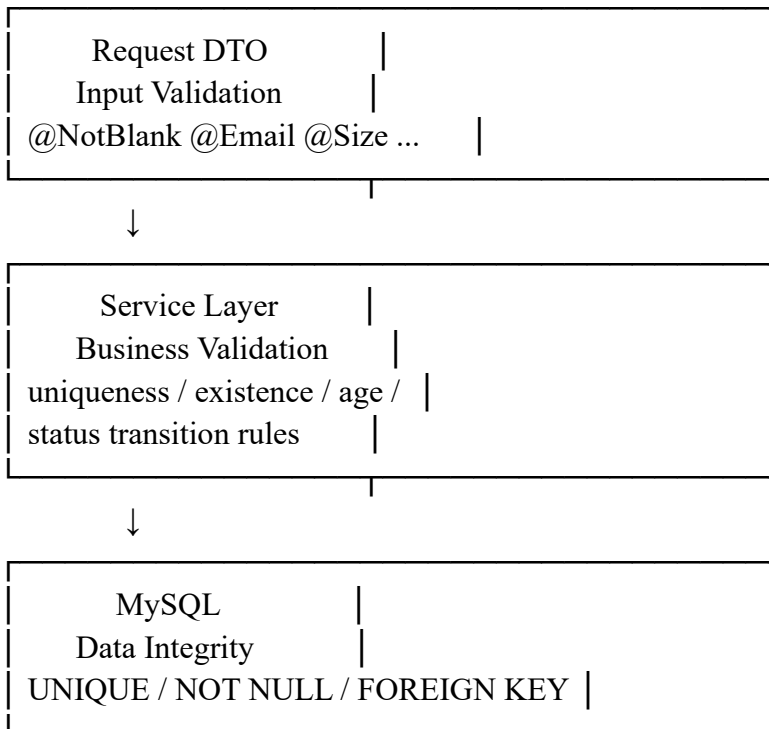
What is the actual business rule for this field?

Then choose the appropriate validation.

Final Validation Implementation Order



Final Architecture



And for retrieval:

Search



Filter



Pagination



Sorting



Parameter Validation



Repository / JPA

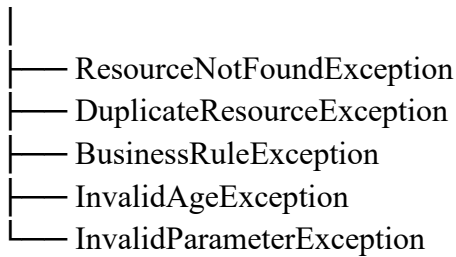
Employee Management System — Exception Handling Parts

1. Global Exception Handler

- `@RestControllerAdvice`
 - `ResponseEntityExceptionHandler`
 - Centralized exception handling
 - No repeated try-catch blocks in controllers
-

2. Custom Exception Hierarchy

`BusinessException`



3. Application Error Codes

Use stable, machine-readable error codes.

Employee

Code	Situation	HTTP
EMP001	Employee not found	404
EMP002	Employee email already exists	409
EMP003	Invalid employee status transition	409
EMP004	Employee below minimum age	400

Department

Code	Situation	HTTP
DEPT001	Department not found	404
DEPT002	Department name already exists	409
DEPT003	Department cannot be deleted because employees exist	409

Request / Framework

Code	Situation	HTTP
REQ001	Validation failed	400
REQ002	Malformed JSON / invalid body value	400
REQ003	Invalid parameter type	400
REQ004	Invalid pagination parameter	400
REQ005	Invalid sorting parameter	400
REQ006	HTTP method not allowed	405
REQ007	Unsupported media type	415
REQ008	URL/resource not found	404

System

Code	Situation	HTTP
SYS001	Database integrity violation	409
SYS002	Unexpected internal server error	500

Security — Phase 7

Code	Situation	HTTP
SEC001	Authentication required/failed	401
SEC002	Access denied	403

4. ErrorCode Enum

Keep all application error codes in one place.

ErrorCode

- EMPLOYEE codes
- DEPARTMENT codes
- REQUEST codes
- SYSTEM codes
- SECURITY codes

Each exception carries its corresponding ErrorCode.

Example:

```
throw new ResourceNotFoundException(  
    ErrorCode.EMPLOYEE_NOT_FOUND,  
    employeeId  
);
```

5. Exception → HTTP Status Mapping

Situation	Exception	Status
Invalid DTO fields	MethodArgumentNotValidException	400
Malformed JSON / invalid enum in body	HttpMessageNotReadableException	400
Invalid path/query type	MethodArgumentTypeMismatchException	400
Invalid pagination/sorting	InvalidParameterException	400
Minimum age violation	InvalidAgeException	400
Resource not found	ResourceNotFoundException	404
Unknown URL	NoResourceFoundException	404
Wrong HTTP method	HttpRequestMethodNotSupportedException	405
Wrong Content-Type	HttpMediaTypeNotSupportedException	415
Duplicate resource	DuplicateResourceException	409
Business rule violation	BusinessRuleException	409
Database constraint violation	DataIntegrityViolationException	409
Unexpected exception	Exception	500
Authentication failure	AuthenticationEntryPoint	401
Access denied	AccessDeniedHandler	403

6. Standard Error Response

Use **one consistent response format**:

ErrorResponse

- |— timestamp
- |— status
- |— code
- |— error
- |— message
- |— path
- |— errors

Example:

```
{
  "timestamp": "2026-09-29T20:30:00",
  "status": 409,
  "code": "EMP002",
  "error": "Conflict",
  "message": "Employee email already exists",
  "path": "/api/employees",
  "errors": []
}
```

Validation example:

```
{
  "timestamp": "2026-09-29T20:30:00",
  "status": 400,
  "code": "REQ001",
  "error": "Bad Request",
  "message": "Validation failed",
  "path": "/api/employees",
  "errors": [
    {
      "field": "email",
      "message": "Email must be valid"
    }
  ]
}
```

7. Spring Framework Exception Handling

Because the handler extends:

ResponseEntityExceptionHandler

customize the appropriate framework methods:

handleMethodArgumentNotValid(...)

handleHttpMessageNotReadable(...)

handleExceptionInternal(...)

This ensures framework errors such as:

400

404

405

415

also use your standard ErrorResponse and ErrorCode.

8. Logging & Monitoring

Error	Log Level	Stack Trace
400	WARN / INFO	✗
404	WARN / INFO	✗
405 / 415	WARN / INFO	✗
409	WARN	✗
500	ERROR	✓

Never expose:

- Stack traces
- SQL/database details
- Credentials
- Internal class/package details

to the client.

9. Implementation Phases

Phase 1 — Department CRUD

Implement:

GlobalExceptionHandler

ErrorCode

ErrorResponse

ResourceNotFoundException

DuplicateResourceException

Validation handling

DataIntegrityViolationException

Phase 2 — Employee Foundation

Reuse the same architecture for:

Create Employee

Get Employee

Get All Employees

Update Employee

Hard Delete

Add:

EMP001

EMP002

Phase 3 — Status + Soft Delete

Add:

BusinessRuleException

EMP003

for invalid employee status transitions.

Phase 4 — Search / Filter / Pagination / Sorting

Add:

InvalidParameterException

REQ003

REQ004

REQ005

Phase 5 — Reports

Reuse existing exception handling.

Phase 6 — Swagger + Testing

Test:

400

404

405

409

415

500

including error codes and response structure.

Phase 7 — Security

Add:

AuthenticationEntryPoint → SEC001 → 401

AccessDeniedHandler → SEC002 → 403

Final Locked Exception Handling

1. **Global Exception Handler**
2. **Custom Exception Hierarchy**
3. **Application Error Codes**
4. **Employee Business Exceptions**
5. **Department Business Exceptions**
6. **Request / Framework Exceptions**
7. **System Exceptions**
8. **ErrorCode Enum**
9. **Standard Error Response**
10. **Spring Framework Exception Handling**
11. **Logging & Monitoring**
12. **Security Exception Handling — Phase 7**

This is the version to keep alongside your **Functional Facilities** → **Technical Facilities** → **Validation** → **Exception Handling** plans.

